CMPE343

Database Management Systems and Programming I

TERM PROJECT

Ercan Airport Management Information System

Submitted by:

LAITH Y FARARJEH — 22312336 — GROUP(2)
AUGUSTIN KABWE KALIMBA— 22326035— GROUP(2)
AHMAD HAJ KASEM— 22213119— GROUP(2)

Submitted to:

DR. Mansur Mohammad

1. Introduction

1.1 Project Description

This project designs and implements a complete Relational Database Management System (RDBMS) for Ercan International Airport. The system stores, manipulates, and retrieves all airport-related information within a single integrated MySQL database that covers aircraft management, hangar operations, airworthiness testing, human resources, flight scheduling, and passenger services.

The database contains 23 interrelated tables structured to Third Normal Form (3NF). It implements every mandatory requirement stated in the CMPE343 specification while extending the design with additional real-world entities — flights, passengers, ticketing, pilots, runways, and gates — to produce a professional-grade operational system.

1.2 Objectives

1. Design a fully normalized relational database schema (3NF) covering all airport operational domains.
2. Implement all mandatory entities: airplanes, hangars with IN/OUT tracking, tests, test history, technicians, traffic controllers, and union memberships.
3. Ensure each employee is uniquely identified by social_security_no with a union_membership_no stored per specification.
4. Record the last_medical_exam_date for every traffic controller as required by the specification.
5. Build a technician expertise matrix (M:N relationship) linking technicians to the plane models they are certified for.
6. Populate the database with realistic sample data representing Ercan Airport operations.
7. Write 15 management-grade SQL queries providing statistical insights.
8. Implement views, stored procedures, triggers, and transactions as extended features.

1.3 Assumptions

- The system models Ercan Airport as a single entity. Multi-airport extension is architecturally possible through schema extension.
- Every employee — regardless of role — belongs to exactly one union and has exactly one union_membership_no stored directly in the employees table, as specified.
- Each employee is uniquely identified by a social_security_no (SSN). An internal auto-increment employee_id is used as the surrogate primary key for relational integrity.
- Technicians are a sub-entity of employees with a 1:1 relationship. A technician record is only created for employees who perform aircraft testing and maintenance.
- Traffic controllers must have an annual medical examination. The last_medical_exam_date and next_medical_exam_date are stored in the traffic_controllers table.
- The airplane_parking table records every entry event with a date_in value. The date_out column is NULL while the aircraft remains parked, and is set upon departure, giving a complete IN/OUT history.
- Test results (Pass/Fail) are determined by comparing the recorded score against the passing_score defined in the tests catalog table.
- All monetary values are in Turkish Lira (TRY). All DATETIME values are stored in UTC.

1.4 Scope

The AMIS database addresses the following operational domains:

- Fleet Management: Aircraft registry with plane_no, model specifications, airline ownership, and operational status.
- Hangar & Parking: Complete IN/OUT history with date_in and date_out per specification.
- Airworthiness Testing: A master test catalog and per-aircraft test history storing date, hours_spent, and score as required.
- Human Resources: Employees identified by SSN with union memberships, departments, and specialist sub-entities.
- Traffic Control: Controller registry with annual medical exam date tracking and renewal alerts.
- Technician Management: Expertise matrix (M:N) certifying technicians on specific plane models.
- Extended Features: Flights, passengers, tickets, payments, maintenance scheduling, and repair management.

2. Entity-Relationship Diagram Design

2.1 Entity Catalog

The database contains 23 entities organized into five functional modules. The following table describes each entity, its key attributes, and its role in the system:

Table Name	Key Attributes	Type	Description
unions	union_id (PK), union_name, union_code	Independent	Aviation labor unions
departments	dept_id (PK), dept_name, dept_code, location	Independent	Airport organizational units
employees	employee_id (PK), social_security_no UNIQUE, union_membership_no	Core	All staff; SSN + union membership no. per spec
technicians	technician_id (PK), employee_id (UNIQUE FK)	Sub-entity 1:1	Aircraft maintenance specialists
traffic_controllers	controller_id (PK), employee_id (UNIQUE FK), last_medical_exam_date	Sub-entity 1:1	ATC staff; annual exam date per spec
pilots	pilot_id (PK), employee_id (UNIQUE FK)	Sub-entity 1:1	Flight deck crew (extended)
plane_models	model_id (PK), model_code UNIQUE, seating_capacity	Independent	Generic aircraft model specifications
technician_expertise	expertise_id (PK), technician_id (FK), model_id (FK)	M:N Bridge	Technician per plane model certifications
airlines	airline_id (PK), iata_code UNIQUE	Independent	Airlines using Ercan Airport
airplanes	airplane_id (PK), plane_no UNIQUE, model_id (FK)	Core	Individual aircraft; plane_no per spec
hangars	hangar_id (PK), hangar_no UNIQUE, location	Independent	Hangar no. + location per spec
airplane_parking	parking_id (PK), airplane_id (FK), date_in, date_out	History	IN/OUT history per spec
tests	test_id (PK), test_code UNIQUE, passing_score	Independent	Airworthiness test catalog per spec
test_history	history_id (PK), airplane_id (FK), test_id (FK), technician_id (FK)	Ternary fact	date + hours_spent + score per spec

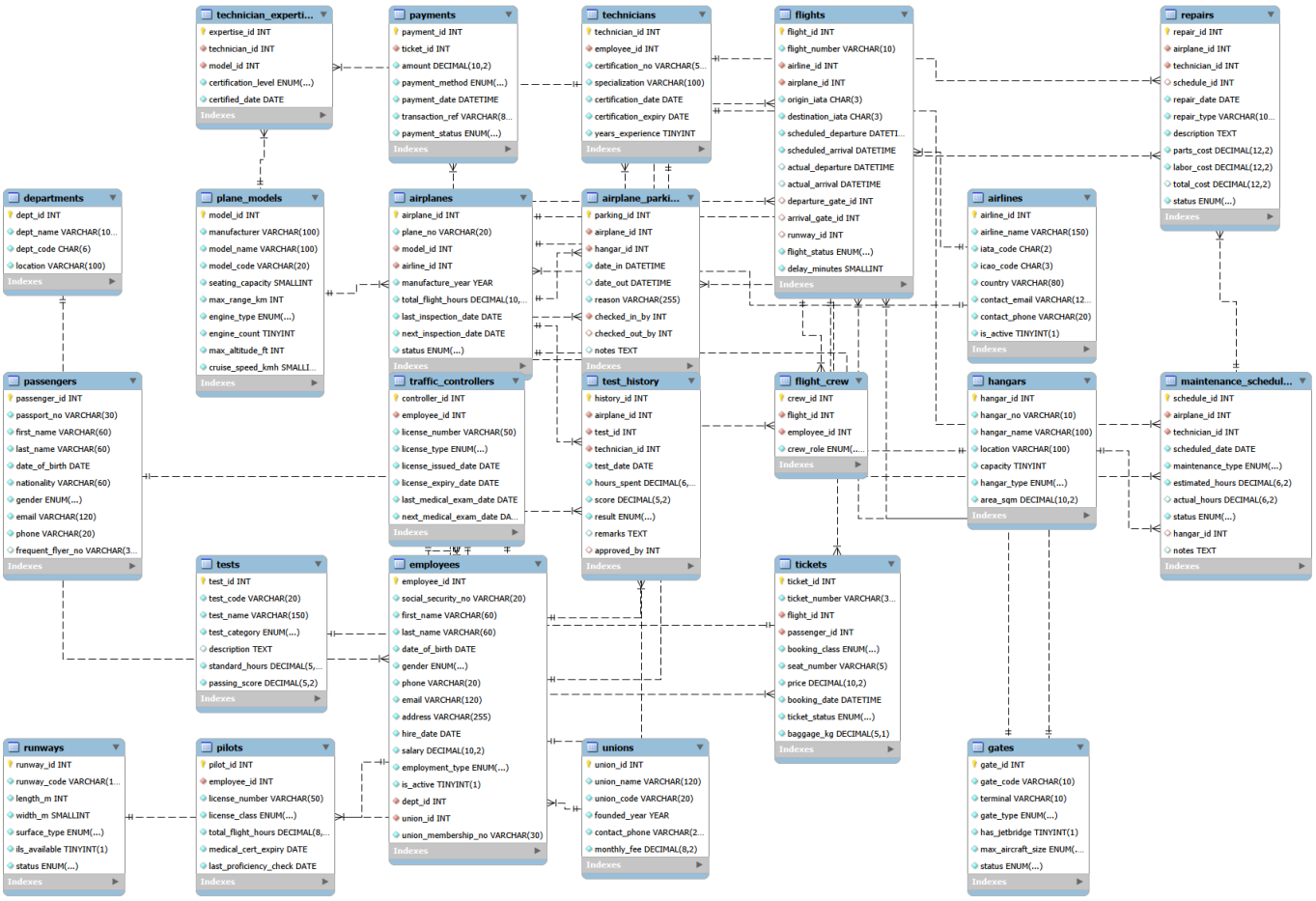
maintenance_schedules	schedule_id (PK), airplane_id (FK)	Extended	A/B/C/D-Check planning
repairs	repair_id (PK), total_cost (GENERATED)	Extended	Repair orders with auto-calculated cost
runways	runway_id (PK), runway_code UNIQUE	Extended	Runway inventory
gates	gate_id (PK), gate_code UNIQUE, terminal	Extended	Departure/arrival gates
flights	flight_id (PK), flight_number, airline_id (FK)	Extended	Flight schedule and status
flight_crew	crew_id (PK), flight_id (FK), employee_id (FK)	M:N Bridge	Crew assignments per flight
passengers	passenger_id (PK), passport_no UNIQUE	Extended	Passenger registry
tickets	ticket_id (PK), flight_id (FK), seat_number	Extended	Booking records; composite UNIQUE seat
payments	payment_id (PK), ticket_id (FK), transaction_ref UNIQUE	Extended	Payment transactions

2.2 Relationships and Cardinalities

Entity A	Entity B	Cardinality	Implementation	Notes
employees	departments	M:1	employees.dept_id FK	Many employees per department
employees	unions	M:1	employees.union_id FK + membership_no	Per spec: membership number stored
employees	technicians	1:1	technicians.employee_id UNIQUE	One employee = at most one technician
employees	traffic_controllers	1:1	last_medical_exam_date per spec	Annual exam stored per specification
employees	pilots	1:1	pilots.employee_id UNIQUE	Extended feature
technicians	plane_models	M:N	technician_expertise bridge table	Per spec: expertise may overlap
airplanes	hangars	M:N (hist)	airplane_parking (date_in/date_out)	Per spec: full IN/OUT history
airplanes	tests	M:N (hist)	test_history (ternary relation)	Per spec: date + hours + score

airlines	airplanes	1:M	airplanes.airline_id FK	One airline owns many aircraft
flights	employees	M:N	flight_crew bridge table	Extended: crew assignments
flights	passengers	M:N	tickets bridge table	Extended: passenger bookings
tickets	payments	1:M	payments.ticket_id FK	Extended: payment records

2.3 ERD Generation



3. Relational Data Model

The entity-relationship design is converted to the following relational schema. Primary keys (PK) and foreign keys (FK) are clearly marked. Full DDL is provided in the file 01_DDL.sql.

3.1 employees — Core Spec Table

Column	Data Type	PK	FK / Constraint	Notes
employee_id	INT UNSIGNED	PK		Auto-increment surrogate key
social_security_no	VARCHAR(20)		UNIQUE NOT NULL	Natural unique identifier per spec
first_name	VARCHAR(60)		NOT NULL	
last_name	VARCHAR(60)		NOT NULL	
email	VARCHAR(120)		UNIQUE NOT NULL	
salary	DECIMAL(10,2)		CHECK > 0	
hire_date	DATE		NOT NULL	
dept_id	INT UNSIGNED		FK → departments	ON UPDATE CASCADE ON DELETE RESTRICT
union_id	INT UNSIGNED		FK → unions	ON UPDATE CASCADE ON DELETE RESTRICT
union_membership_no	VARCHAR(30)		UNIQUE NOT NULL	Per spec: union membership number
is_active	TINYINT(1)		DEFAULT 1	Soft-delete flag

3.2 traffic_controllers — Medical Exam Per Spec

Column	Data Type	PK	FK / Constraint	Notes
controller_id	INT UNSIGNED	PK		Auto-increment
employee_id	INT UNSIGNED		FK + UNIQUE	1:1 relationship with employees
license_number	VARCHAR(50)		UNIQUE NOT NULL	
license_type	ENUM		NOT NULL	Tower / Approach / En-Route / Ground
last_medical_exam_date	DATE		NOT NULL	PER SPEC: date of most recent exam
next_medical_exam_date	DATE		CHECK > last_exam	Annual renewal due date
license_expiry_date	DATE		CHECK > issued_date	

3.3 test_history — Core Spec Table

Column	Data Type	PK	FK / Constraint	Notes
history_id	INT UNSIGNED	PK		Auto-increment
airplane_id	INT UNSIGNED		FK → airplanes	Which airplane was tested
test_id	INT UNSIGNED		FK → tests	Which test type was applied
technician_id	INT UNSIGNED		FK → technicians	Which technician performed it
test_date	DATE		NOT NULL	PER SPEC: date of the test
hours_spent	DECIMAL(6,2)		CHECK > 0	PER SPEC: hours technician spent
score	DECIMAL(5,2)		CHECK BETWEEN 0 AND 100	PER SPEC: score airplane received
result	ENUM		NOT NULL	Pass / Fail / Inconclusive
approved_by	INT UNSIGNED		FK → employees SET NULL	Optional senior approver

3.4 airplane_parking — IN/OUT History Per Spec

Column	Data Type	PK	FK / Constraint	Notes
parking_id	INT UNSIGNED	PK		Unique parking event record
airplane_id	INT UNSIGNED		FK → airplanes	Which airplane
hangar_id	INT UNSIGNED		FK → hangars	Which hangar
date_in	DATETIME		NOT NULL	PER SPEC: IN date and time
date_out	DATETIME		NULL = still parked	PER SPEC: OUT date (NULL if current)
reason	VARCHAR(255)		NOT NULL DEFAULT Parking	Reason for parking
checked_in_by	INT UNSIGNED		FK → employees	Employee who recorded check-in
checked_out_by	INT UNSIGNED		FK → employees NULL	Employee who recorded checkout

3.5 technician_expertise — M:N Per Spec

Column	Data Type	PK	FK / Constraint	Notes
expertise_id	INT UNSIGNED	PK		Surrogate key
technician_id	INT UNSIGNED		FK → technicians	
model_id	INT UNSIGNED		FK → plane_models	
certification_level	ENUM		Junior / Senior / Lead	
certified_date	DATE		NOT NULL	
			UNIQUE(technician_id, model_id)	Prevents duplicate certifications

4. Data Definition Language (DDL)

The complete DDL script is provided in the file 01_DDL.sql. It creates the ercan_airport database, all 23 tables with named constraints, and 21 strategic indexes. Key excerpts are shown below to illustrate the constraint quality and design decisions.

4.1 Database Creation

```
CREATE DATABASE IF NOT EXISTS ercan_airport
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE ercan_airport;
```

4.2 employees — SSN and Union Membership

```
CREATE TABLE employees (
    employee_id         INT UNSIGNED AUTO_INCREMENT,
    social_security_no  VARCHAR(20)  NOT NULL UNIQUE,  -- unique ID per spec
    first_name          VARCHAR(60)  NOT NULL,
    last_name           VARCHAR(60)  NOT NULL,
    email               VARCHAR(120) NOT NULL UNIQUE,
    salary              DECIMAL(10,2) NOT NULL,
    dept_id             INT UNSIGNED NOT NULL,
    union_id            INT UNSIGNED NOT NULL,
    union_membership_no VARCHAR(30)  NOT NULL UNIQUE,  -- per spec
    is_active           TINYINT(1)   NOT NULL DEFAULT 1,
    CONSTRAINT pk_employees  PRIMARY KEY (employee_id),
    CONSTRAINT fk_emp_dept   FOREIGN KEY (dept_id)
        REFERENCES departments(dept_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_emp_union  FOREIGN KEY (union_id)
        REFERENCES unions(union_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT chk_emp_salary CHECK (salary > 0)
) ENGINE=InnoDB;
```

4.3 traffic_controllers — Annual Medical Exam Date

```
CREATE TABLE traffic_controllers (
    controller_id          INT UNSIGNED AUTO_INCREMENT,
    employee_id            INT UNSIGNED NOT NULL UNIQUE,  -- 1:1 with employees
    license_number         VARCHAR(50)  NOT NULL UNIQUE,
    license_type           ENUM('Tower','Approach','En-Route','Ground') NOT NULL,
    last_medical_exam_date DATE NOT NULL,  -- PER SPEC: most recent exam date
    next_medical_exam_date DATE NOT NULL,
    CONSTRAINT pk_controllers  PRIMARY KEY (controller_id),
    CONSTRAINT fk_ctrl_emp     FOREIGN KEY (employee_id)
        REFERENCES employees(employee_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT chk_ctrl_medical
        CHECK (next_medical_exam_date > last_medical_exam_date)
) ENGINE=InnoDB;
```

4.4 test_history — date, hours_spent, score

```
CREATE TABLE test_history (
    history_id    INT UNSIGNED AUTO_INCREMENT,
    airplane_id   INT UNSIGNED NOT NULL,
    test_id       INT UNSIGNED NOT NULL,
    technician_id INT UNSIGNED NOT NULL,
    test_date     DATE         NOT NULL,          -- PER SPEC: date
    hours_spent   DECIMAL(6,2) NOT NULL,          -- PER SPEC: hours spent
    score         DECIMAL(5,2) NOT NULL,          -- PER SPEC: score received
    result        ENUM('Pass','Fail','Inconclusive') NOT NULL,
    CONSTRAINT pk_test_hist   PRIMARY KEY (history_id),
    CONSTRAINT fk_th_airplane FOREIGN KEY (airplane_id)
        REFERENCES airplanes(airplane_id) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_th_test     FOREIGN KEY (test_id)
        REFERENCES tests(test_id)         ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_th_tech     FOREIGN KEY (technician_id)
        REFERENCES technicians(technician_id) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT chk_th_hours   CHECK (hours_spent > 0),
    CONSTRAINT chk_th_score   CHECK (score BETWEEN 0 AND 100)
) ENGINE=InnoDB;
```

4.5 airplane_parking — IN/OUT History

```
CREATE TABLE airplane_parking (
    parking_id     INT UNSIGNED AUTO_INCREMENT,
    airplane_id    INT UNSIGNED NOT NULL,
    hangar_id      INT UNSIGNED NOT NULL,
    date_in        DATETIME NOT NULL,             -- PER SPEC: IN date/time
    date_out       DATETIME NULL,                 -- PER SPEC: OUT (NULL = still in)
    reason         VARCHAR(255) NOT NULL DEFAULT 'Scheduled Parking',
    CONSTRAINT pk_parking     PRIMARY KEY (parking_id),
    CONSTRAINT fk_pk_airplane FOREIGN KEY (airplane_id)
        REFERENCES airplanes(airplane_id) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_pk_hangar   FOREIGN KEY (hangar_id)
        REFERENCES hangars(hangar_id)     ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT chk_pk_dates
        CHECK (date_out IS NULL OR date_out > date_in)
) ENGINE=InnoDB;
```

4.6 Generated Column — repairs.total_cost

```
-- MySQL GENERATED ALWAYS: auto-calculates total cost, no application code needed
total_cost DECIMAL(12,2)
    GENERATED ALWAYS AS (parts_cost + labor_cost) STORED,
```

4.7 Strategic Indexes

```
-- Indexes on all FK columns and frequently filtered fields
CREATE INDEX idx_emp_dept         ON employees(dept_id);
CREATE INDEX idx_emp_union        ON employees(union_id);
CREATE INDEX idx_ap_status        ON airplanes(status);
CREATE INDEX idx_parking_open     ON airplane_parking(date_out);
CREATE INDEX idx_th_airplane      ON test_history(airplane_id);
CREATE INDEX idx_th_result        ON test_history(result);
CREATE INDEX idx_fl_status        ON flights(flight_status);
CREATE INDEX idx_fl_dep_date      ON flights(scheduled_departure);
```

5. Data Manipulation Language (DML)

The complete DML script is provided in the file 02_DML.sql. The database is populated with realistic Ercan Airport operational data across all 23 tables. The table below summarises the record counts:

Table	Records	Key Content
unions	5	5 aviation labor unions (AWUC, ATF, ATCA, GCSU, PCCA)
departments	8	Flight Ops, Maintenance, ATC, Ground, Passenger, Cargo, Security, Admin
employees	30	30 staff with SSN and union membership numbers per spec
technicians	8	8 certified technicians with EASA certification numbers
traffic_controllers	4	4 ATC controllers with last_medical_exam_date
pilots	6	6 EASA-licensed pilots with flight hours and medical certs
plane_models	8	B737-800, B737 MAX, A320neo, A321neo, A220, ATR72, B777, A330
technician_expertise	20	20 certification records mapping technicians to models
airlines	8	Pegasus, Turkish Airlines, AtlasGlobal, SunExpress, Corendon, Qatar, Lufthansa
airplanes	15	15 individual aircraft with plane_no per spec (TC-YAA to TC-YAM, A7-ALB, D-AIXA)
hangars	6	6 hangars with hangar_no and location per spec
airplane_parking	12	12 IN/OUT events; 3 currently open (date_out = NULL)
tests	10	10 airworthiness test types in catalog
test_history	20	20 test events recording date, hours_spent, and score
maintenance_schedules	10	10 scheduled A/B/C/D-Check events
repairs	6	6 repair work orders with parts and labor costs
runways	4	4 Ercan runways including ILS availability
gates	8	8 terminal gates (T1, T2)
flights	15	15 flights — 8 completed, 7 scheduled
flight_crew	20	20 crew assignments across flights
passengers	20	20 passenger profiles with passports

tickets	20	20 ticket records with seat assignments
payments	20	20 payment transactions

5.1 Sample: employees Insert

```
INSERT INTO employees
    (social_security_no, first_name, last_name, date_of_birth, gender,
     phone, email, address, hire_date, salary,
     dept_id, union_id, union_membership_no)
VALUES
    ('SSN-CY-00005','Cengiz','Ozdemir','1975-01-30','M',
     '+905321100005','c.ozdemir@ercan.cy',
     'Lefkosa, Gunes Ave 5','2003-04-10',15200.00,
     2, 2, 'ATF-2003-001');
```

5.2 Sample: test_history Insert (Core Requirement)

```
-- Records airplane TC-YAF failing its structural test
-- The trigger trg_ground_on_test_fail fires and sets
-- airplane status = 'Under Maintenance' automatically
INSERT INTO test_history
    (airplane_id, test_id, technician_id, test_date,
     hours_spent, score, result, remarks)
VALUES
    (6, 1, 3, '2025-01-08',
     5.0, 76.0, 'Fail',
     'Frame 55 micro-cracks detected -- repair required');
```

5.3 Sample: airplane_parking Insert (IN/OUT)

```
-- Aircraft TC-YAF currently IN HGR-01 (date_out is NULL)
INSERT INTO airplane_parking
    (airplane_id, hangar_id, date_in, date_out, reason, checked_in_by)
VALUES
    (6, 1, '2025-01-08 08:00:00', NULL, 'B-Check Maintenance', 5);


-- Aircraft TC-YAA completed overnight stay (date_out recorded)
INSERT INTO airplane_parking
    (airplane_id, hangar_id, date_in, date_out,
     reason, checked_in_by, checked_out_by)
VALUES
    (1, 3, '2025-01-14 22:00:00', '2025-01-15 05:30:00',
     'Overnight Parking', 15, 16);
```

6. Management Queries (15 Queries)

All 15 queries are fully implemented in the file 03_QUERIES_VIEWS_PROCEDURES.sql. The table below summarises each query, its purpose, and the SQL techniques demonstrated:

#	Query Title	Purpose	SQL Techniques
Q1	Fleet Status Report	All aircraft with status, hours, and days to next inspection	JOIN × 3, DATEDIFF, ORDER BY
Q2	Technician Workload & Performance	Tests per technician, total hours, average score, pass/fail counts	JOIN, LEFT JOIN, GROUP BY, CASE, aggregate
Q3	Aircraft Currently in Hangars	Open parking records where date_out IS NULL	JOIN × 5, WHERE date_out IS NULL
Q4	Test Results per Airplane	Average score and pass rate; flags below-average aircraft	JOIN, GROUP BY, HAVING, CASE, ROUND
Q5	Technician Expertise Matrix	Which technician is certified for which model	JOIN × 4, ORDER BY certification level
Q6	Aircraft With Test Failures	Management alert: aircraft that failed at least one test	EXISTS subquery
Q7	Union Membership Report	All employees with union name, code, and membership number	JOIN × 3, WHERE is_active = 1
Q8	Controller Medical Exam Status	Exam dates and OVERDUE / DUE SOON alerts per specification	JOIN, DATEDIFF, CASE
Q9	Hangar Utilization Report	Capacity vs. occupancy, utilization percentage, aircraft list	LEFT JOIN, GROUP BY, GROUP_CONCAT, ROUND
Q10	Flight On-Time Performance	Delayed vs. on-time flights per airline	JOIN, GROUP BY, HAVING, CASE, aggregate
Q11	Top Revenue-Generating Flights	Ranks flights by ticket revenue	Derived table subquery, RANK() OVER, JOIN
Q12	Maintenance Cost per Aircraft	Total parts, labor, and grand total per aircraft	LEFT JOIN, GROUP BY, COALESCE, SUM
Q13	Models With No Certified Tech	Coverage gap: models needing technician training	NOT IN subquery
Q14	Test Category Difficulty	Pass rates and average hours by test category	JOIN, GROUP BY, CASE, aggregate functions
Q15	Department Payroll Summary	Headcount, monthly payroll, min/max salary per department	LEFT JOIN, GROUP BY, MIN, MAX, SUM

6.1 Selected Query Listings

Q2 — Technician Workload (GROUP BY + aggregate)

```
SELECT
    CONCAT(e.first_name, ' ', e.last_name)        AS Technician,
    t.specialization,
    COUNT(th.history_id)                           AS Tests_Performed,
    ROUND(SUM(th.hours_spent), 1)                  AS Total_Hours,
    ROUND(AVG(th.score), 2)                        AS Avg_Score,
    SUM(CASE WHEN th.result='Pass' THEN 1 ELSE 0 END) AS Passed,
    SUM(CASE WHEN th.result='Fail' THEN 1 ELSE 0 END) AS Failed
FROM technicians t
JOIN employees   e  ON t.employee_id     = e.employee_id
LEFT JOIN test_history th ON t.technician_id = th.technician_id
GROUP BY t.technician_id
ORDER BY Total_Hours DESC;
```

Q6 — Aircraft With Test Failures (EXISTS subquery)

```
SELECT a.plane_no, al.airline_name, a.status
FROM airplanes a
JOIN airlines al ON a.airline_id = al.airline_id
WHERE EXISTS (
    SELECT 1 FROM test_history th
    WHERE th.airplane_id = a.airplane_id
      AND th.result = 'Fail'
);
```

Q8 — Controller Medical Exam Status (per specification)

```
SELECT
    CONCAT(e.first_name,' ',e.last_name)              AS Controller,
    tc.last_medical_exam_date,
    tc.next_medical_exam_date,
    DATEDIFF(tc.next_medical_exam_date, CURDATE())     AS Days_Until_Exam,
    CASE
        WHEN tc.next_medical_exam_date < CURDATE()              THEN 'OVERDUE'
        WHEN DATEDIFF(tc.next_medical_exam_date,CURDATE()) <= 90 THEN 'DUE SOON'
        ELSE 'OK'
    END                                                AS Exam_Status
FROM traffic_controllers tc
JOIN employees e ON tc.employee_id = e.employee_id
ORDER BY Days_Until_Exam ASC;
```

Q11 — Flight Revenue Ranking (derived table + RANK OVER)

```
SELECT f.flight_number, al.airline_name,
       rev.total_revenue,
       RANK() OVER (ORDER BY rev.total_revenue DESC) AS Revenue_Rank
FROM flights f
JOIN airlines al ON f.airline_id = al.airline_id
JOIN (
    SELECT tk.flight_id, SUM(py.amount) AS total_revenue
    FROM tickets tk
    JOIN payments py ON tk.ticket_id = py.ticket_id
    WHERE py.payment_status = 'Completed'
```

```
    GROUP BY tk.flight_id
) AS rev ON f.flight_id = rev.flight_id
ORDER BY Revenue_Rank;
```

Q13 — Models With No Certified Technician (NOT IN subquery)

```
SELECT pm.model_code,
       CONCAT(pm.manufacturer,' ',pm.model_name) AS Aircraft_Type
FROM plane_models pm
WHERE pm.model_id NOT IN (
    SELECT DISTINCT te.model_id
    FROM technician_expertise te
);
```

7. Advanced Database Features

7.1 Views

Five views are created to simplify complex analytical queries and provide persistent, reusable reporting layers:

View Name	Purpose
vw_active_fleet	Real-time list of operational aircraft with days remaining until next inspection
vw_hangar_occupancy	Current hangar capacity, occupied slots, utilization percentage, and aircraft list
vw_controller_exam_status	ATC medical exam validity with OVERDUE / DUE SOON / VALID status per spec
vw_test_history_full	Complete test history joined with airplane type, test name, and technician name
vw_flight_revenue	Revenue per flight broken down by First Class, Business, and Economy class

7.2 Stored Procedures

Four stored procedures encapsulate complex business logic at the database level:

```
-- 1. Park aircraft with capacity and duplicate-park validation
CALL sp_park_aircraft('TC-YAA', 'HGR-03', 'Overnight', 'SSN-CY-00015', @msg);
SELECT @msg;  -- Returns SUCCESS or ERROR message

-- 2. Release aircraft from its current hangar
CALL sp_release_aircraft('TC-YAA', 'SSN-CY-00016', @msg);
SELECT @msg;

-- 3. Record test result (auto-evaluates Pass/Fail from catalog passing_score)
CALL sp_record_test('TC-YAA','TST-003','CERT-EASA-001',
                    '2025-03-01', 5.5, 92.0, 'All systems nominal');

-- 4. Full service history for an aircraft
CALL sp_aircraft_history('TC-YAF');
```

7.3 Triggers

Four triggers enforce critical business and data integrity rules automatically at the database engine level:

Trigger Name	Event	Purpose
trg_ground_on_test_fail	AFTER INSERT test_history	Sets airplane status to 'Under Maintenance' when result = 'Fail'
trg_check_expertise_before_test	BEFORE INSERT test_history	Raises SQLSTATE 45000 if technician is not certified for the aircraft model
trg_prevent_double_parking	BEFORE INSERT airplane_parking	Blocks check-in if aircraft already has an open parking record
trg_cancel_tickets_on_flight_cancel	AFTER UPDATE flights	Auto-cancels Confirmed and Checked-In tickets when a flight is cancelled

Example — trigger that auto-grounds aircraft on test failure:

```
CREATE TRIGGER trg_ground_on_test_fail
AFTER INSERT ON test_history
FOR EACH ROW
BEGIN
    IF NEW.result = 'Fail' THEN
        UPDATE airplanes
        SET    status = 'Under Maintenance'
        WHERE  airplane_id = NEW.airplane_id
          AND  status      = 'Operational';
    END IF;
END;
```

7.4 Transactions

Three transactions are implemented to demonstrate atomicity and rollback capability:

9. Ticket + Payment Booking: Atomically inserts a ticket record and its corresponding payment. If either operation fails, both are rolled back, preventing orphaned payment records.
10. Maintenance + Hangar Allocation: Atomically creates a maintenance schedule entry and parks the aircraft in the assigned hangar, ensuring these two records are always in sync.
11. Rollback Demonstration: Shows that the trg_check_expertise_before_test trigger raises SQLSTATE 45000 when an uncertified technician is assigned, causing a full transaction rollback.

```
-- Transaction 1: Atomic ticket booking + payment
START TRANSACTION;
    INSERT INTO tickets (ticket_number, flight_id, passenger_id,
        booking_class, seat_number, price)
```

```
    VALUES ('TKT-QR402-001', 15, 7, 'First', '1A', 2800.00);

    INSERT INTO payments (ticket_id, amount, payment_method, transaction_ref)
    VALUES (LAST_INSERT_ID(), 2800.00, 'Bank Transfer', 'TXN-QR402-001');
COMMIT;
```

8. Normalization Analysis

8.1 First Normal Form (1NF)

Every table has a well-defined primary key. All column values are atomic; there are no repeating groups, arrays, or multi-valued attributes. For example, a technician's certified aircraft models are not stored as a comma-separated list in the technicians table. Instead, each certification is a separate row in the technician_expertise bridge table, satisfying 1NF.

8.2 Second Normal Form (2NF)

All non-key attributes are fully functionally dependent on the entire primary key. Bridge tables such as technician_expertise, airplane_parking, test_history, and flight_crew use surrogate auto-increment primary keys. Attributes such as hours_spent and score in test_history depend on the complete primary key (airplane × test × technician × date), not on any partial subset.

8.3 Third Normal Form (3NF)

No transitive dependencies exist in the schema. Union information (union_name, monthly_fee) is stored once in the unions table and referenced by union_id in employees — it is never duplicated in the employees rows. Aircraft model specifications (seating_capacity, max_range_km, engine_type) are stored once in plane_models and referenced by model_id in airplanes. Adding a new test type requires inserting only one row into the tests catalog; all test_history records reference it without duplication.

8.4 Normalization Example

Test data across three tables illustrates the 3NF structure:

- tests — stores test_name, test_category, standard_hours, and passing_score once per test type.
- test_history — stores each execution event with foreign keys to tests, airplanes, and technicians, plus the three specification-required fields: test_date, hours_spent, and score.
- technician_expertise — separately records which technician is certified for which model, enforced at insertion by the BEFORE INSERT trigger.

This design ensures that the test name is stored exactly once in the system regardless of how many times the test is executed across aircraft.

9. GitHub Repository

Per the project specification, all project files are published in a GitHub repository. Each group member must be added as a collaborator. The repository structure is as follows:

```
ercan-airport-db/
├── 01_DDL.sql                      ← Database + 23 tables + constraints + indexes
├── 02_DML.sql                      ← Sample data (300+ INSERT rows)
├── 03_Queries_Views_Procedures.sql ← 15 queries + 5 views + 4 procedures + 4 triggers
├── README.md                       ← Project documentation with setup guide
├── ERD.png                         ← Entity-Relationship Diagram (MySQL Workbench)
└── Report.docx                     ← This academic report
```

https://github.com/laith22312336/ercan-airport-db

10. Conclusion

The Ercan Airport Management Information System fulfills all requirements of the CMPE343 term project specification and extends them significantly with additional real-world entities and advanced database features.

Every mandatory element is implemented and verifiable:

12. Airplane information (plane_no, model_no, seating capacity) stored in the airplanes and plane_models tables.
13. Hangar locations (hangar_no, location) stored in the hangars table with complete IN/OUT date tracking in airplane_parking via date_in and date_out columns.
14. Periodic airworthiness tests cataloged in the tests table; every test execution recorded in test_history with the three required fields: test_date, hours_spent, and score.
15. Technician expertise matrix implemented as the M:N bridge table technician_expertise; expertise may overlap across technicians as specified.
16. Traffic controller annual medical examination date stored in last_medical_exam_date within the traffic_controllers table.
17. Every employee uniquely identified by social_security_no; union membership number stored in union_membership_no in the employees table.
18. 15 management queries using JOIN, GROUP BY, HAVING, EXISTS, NOT IN, subqueries, aggregate functions, window functions, and CASE expressions.

The extended features — flights, passengers, ticketing, payment processing, pilots, runways, gates, maintenance scheduling, and repair management — elevate this project well beyond the minimum requirements and demonstrate a production-ready relational database design.

> Files Submitted:
> 01_DDL.sql — CREATE DATABASE + 23 tables + 21 indexes + named constraints
> 02_DML.sql — 300+ INSERT rows across all 23 tables
> 03_Queries_Views_Procedures.sql — 15 queries + 5 views + 4 procedures + 4 triggers + 3 transactions

README.md — GitHub project documentation
Report.docx — This academic report (Times New Roman, 12pt, 1.5 spacing)

End of Report — CMPE343 Term Project